JAVA RUNTIME & OOP FOUNDATIONS • QA TO DEVELOPER ROADMAP

Java Architecture & Object-Oriented Programming

From JVM Mechanics to Modern Type Systems, Formal Definitions, and Production Code

Runtime Mechanics

OOP Core Pillars

Contracts & Verification

CONCEPT 01

What is OOP?

Paradigm Foundation

SIMPLE DEFINITION

Object-Oriented Programming (OOP) is a way of designing software by packaging related data (variables) and actions (methods) together into reusable building blocks called "objects".

REAL-WORLD ANALOGY

Physical Bank Branch

A bank branch does not keep loose cash and ledgers on the sidewalk. Money is secured inside cash registers and vaults. Customers interact through tellers via explicit procedures: deposit, withdraw, and verify balance.

State: Vault cash, ledger accounts.

Behavior: Deposit slip processing, balance inquiries.

QA VIEW & BENEFIT

Encapsulates state behind authorized procedures. Prevents external callers from setting invalid balances.

BankBranch.java — Code Implementation

```
public class BankBranch {
private BigDecimal vaultBalance; // State

public void processDeposit(BigDecimal amount) {
if (amount.compareTo(BigDecimal.ZERO) ⩽ 0) {
throw new IllegalArgumentException(
"Positive deposit required"
);
}
this.vaultBalance = this.vaultBalance.add(amount);
}
}
```

CONCEPT 02

Why OOP?

Maintainability & Scale

SIMPLE DEFINITION

Without OOP, code turns into a messy bowl of spaghetti where fixing one bug breaks five other things. OOP organizes code into neat, independent sections so you can fix, test, or add features without breaking the rest of the application.

REAL-WORLD ANALOGY

Modular Audio Systems

In old monolithic audio players, if the cassette deck broke, the entire stereo unit was ruined. In modern modular hi-fi rigs, each component connects through standardized cables. You can replace a speaker without rewiring the amplifier.

PROCEDURAL FLAW

Global state mutations trigger unpredictable bugs in unrelated modules.

QA VIEW & BENEFIT

Decoupled components allow localized fixes. Add new fee tiers without modifying tested legacy code.

Procedural vs. OOP Code

```
// PROCEDURAL ANTI-PATTERN (High Regression Risk)
double fee = (type == "VIP") ? 0.0 : amount * 0.02;

// OOP PATTERN (Open-Closed Principle)
public interface FeeStrategy {
  BigDecimal compute(BigDecimal amount);
}

public class StandardFee implements FeeStrategy { ... }
public class VipFee implements FeeStrategy { ... }
```

Runtime Mechanics

JDK vs. JRE vs. JVM

Java Execution Layers

Simple Definition

The JDK is the developer toolbox for writing and compiling code. The JRE is the package containing the libraries needed to run Java programs. The JVM is the actual virtual computer engine that runs your compiled code line-by-line.

LAYER 1: DEVELOPER TOOLS

JDK

Contains the compiler (javac), debuggers (jdb), and archival tools (jar). Needed by engineers to write and build software.

Real-World: A chef's entire professional kitchen setup with knives and ovens.

Produces .class bytecode files.

LAYER 2: RUNTIME SUPPORT

JRE

Bundles the JVM with the core Java Class Library (e.g., java.lang, java.util) necessary to run precompiled applications.

Real-World: The dining table setting and cutlery for eating the meal.

Supplies standard libraries for runtime.

LAYER 3: VIRTUAL ENGINE

JVM

The execution engine. Manages class loading, garbage collection, and Just-In-Time (JIT) compilation to native CPU machine instructions.

Real-World: The engine inside the vehicle that actually turns wheels.

Executes bytecode on physical hardware.

COMPILATION PIPELINE

Bytecode: Write Once, Run Anywhere

Platform Neutrality

SIMPLE DEFINITION

Bytecode is an intermediate universal language (saved in .class files). Instead of turning human code directly into Windows or Mac instructions, Java compiles it into Bytecode, which any JVM anywhere can read and run.

REAL-WORLD ANALOGY

Sheet Music

A musical score is written in standardized notation (staves, notes, clefs). It is completely independent of the instrument. A grand piano, an acoustic guitar, or a synthesizer can all read the identical sheet music and translate it into instrument-specific acoustic sounds.

PORTABLE INSTRUCTION SET

Java Code: Composer's concept → **Bytecode:** Standard sheet music → **JVM:** Instrument performing the piece.

QA VIEW & BENEFIT

Build once in CI/CD pipeline, deploy identically to any server across Linux, macOS, and Windows.

Compilation to Bytecode View

```
// JAVA SOURCE (PaymentService.java)
public int calculateTax(int price) {
  return price * 18 / 100;
}

// COMPILED BYTECODE (javap -c output)
0: iload_1
1: bipush 18
3: imul
4: bipush 100
6: idiv
7: ireturn
```

OOP CORE ELEMENT

What is an Object?

Heap Memory Instance

SIMPLE DEFINITION

An Object is a real, living item created in computer memory. It has its own unique information (state, like a person's name and balance) and actions it can perform (behavior, like depositing money).

REAL-WORLD ANALOGY

Physical Credit Card

A physical plastic card in your wallet is an object. It holds specific state: Card Number "4532...", Cardholder "Alex Vance", and Expiration Date "08/29". It has actions: swipe, tap, freeze card.

DISTINCT IDENTITY

Even if two cards have identical credit limits, they are distinct physical objects with unique chip serial numbers.

QA VIEW & ISOLATION

Objects represent tangible entities in memory. QA verifies isolated execution across concurrent user sessions—modifying state in one object never mutates another.

Code Implementation (CreditCard.java)

```java
// 1. Instantiating independent objects in heap memory
CreditCard card1 = new CreditCard("41112222", "Alex");
CreditCard card2 = new CreditCard("55008888", "Maya");

// 2. Each object manages its own distinct state
card1.charge(new BigDecimal("150.00"));

// 3. Verify state isolation across instances
System.out.println(card1.getBalance()); // → 150.00
System.out.println(card2.getBalance()); // → 0.00
```

OOP CORE ELEMENT

What is a Class?

Architectural Template

SIMPLE DEFINITION

A Class is simply the master template or blueprint that describes what kind of data an object will hold and what actions it will be able to do once it is created.

REAL-WORLD ANALOGY

Architectural Blueprint

An architectural blueprint specifies room dimensions, electrical circuits, and plumbing pipes. You cannot sleep inside a blueprint; it takes zero physical land. From one blueprint, builders can construct 500 identical houses in a development.

BLUEPRINT VS INSTANCE

Class: Blueprint definition → **Object:** Actual physical house built on the street.

QA VIEW & SCHEMA ASSERTIONS

The class dictates what fields must be tested in API assertions. QA tests schema validation to ensure instantiated object payloads strictly adhere to declared attribute types and operational contracts.

Class Template Definition (UserAccount.java)

```java
// Master class blueprint describing state & behavior
public class UserAccount {

   // Structural attributes (State schema)
   private final String userId;
   private String email;
   private boolean kycVerified;

   // Behavioral methods (Operations schema)
   public void approveKyc() {
      this.kycVerified = true;
   }

}
```

PILLAR 01

Encapsulation

Data Hiding & Invariants

SIMPLE DEFINITION

Encapsulation means keeping your data variables private and protected so outsiders cannot directly mess with them. They must use safe, public methods that check the rules first.

REAL-WORLD ANALOGY

Medicine Capsule

A medicinal capsule seals active pharmaceutical ingredients inside a gelatin shell. Patients take the exact prescribed dosage without manually handling or contaminating the medicinal powder inside.

WITHOUT ENCAPSULATION

Direct public access allows bad actors or accidental typos to set an account balance to -$9,999.00.

QA VIEW & INVARIANTS

Protects internal state from illegal external tampering. Validation logic ensures impossible states cannot occur.

Code Implementation (Wallet.java)

```java
// 1. Encapsulated state guarded by private access
public class Wallet {
  private BigDecimal balance = BigDecimal.ZERO;

  // 2. Guarded public method enforces rules
  public void withdraw(BigDecimal amount) {
    if (amount.compareTo(BigDecimal.ZERO) <= 0) {
      throw new IllegalArgumentException(
        "Amount must be positive");
    }
    if (balance.compareTo(amount) < 0) {
      throw new InsufficientFundsException(
        "Balance too low");
    }
    this.balance = this.balance.subtract(amount);
  }
}
```

PILLAR 02

Inheritance: Types & Mechanics

Subtyping & Reuse

SIMPLE DEFINITION

Inheritance allows a new child class to automatically reuse the fields and methods of an existing parent class, saving you from writing duplicate code.

ALL 4 INHERITANCE FLAVORS

- **Single:** Class B extends Class A (e.g., SavingsAccount extends Account)
- **Multilevel:** C extends B, B extends A (e.g., FixedDeposit → Savings → Account)
- **Hierarchical:** B and C both extend A (e.g., Checking and Savings extend Account)
- **Multiple (via Interfaces):** Class A implements Auditable, Taxable (solves the Deadly Diamond Problem)

REAL-WORLD ANALOGY

A smartphone is a telephone; it adds touchscreens and cameras while inheriting cellular calls.

QA VIEW & AUDIT REUSE

Audit logging verified once on base superclass; automatically inherited by all account types.

FinTech Hierarchy Code (Account.java)

```java
// Base Superclass
public class Account {
   protected String accountId;
   public void logAudit() {
      // Centralized audit trail logic
   }
}

// Single + Hierarchical Extension
public class SavingsAccount extends Account {
   private BigDecimal interestRate =
      new BigDecimal("0.04");

   public void applyInterest() {
      // Interest calculation logic
   }
}
```

PILLAR 03

Abstraction: Interfaces vs. Abstract Classes

Contract vs. Template

SIMPLE DEFINITION

Abstraction hides the complicated inner workings and shows only what is necessary to the user. An Interface is a strict list of rules (what must be done), while an Abstract Class is a half-finished class that shares common code.

INTERFACE

Pure Contract

A wall power socket defines an interface: two pins, 230 volts. It doesn't care whether you plug in a laptop, toaster, or phone charger.

PaymentGateway.java

```java
// 1. Pure interface contract
public interface PaymentGateway {
   PaymentResult charge(BigDecimal amt);
}

// 2. Concrete implementation
public class StripeGateway implements PaymentGateway {
   public PaymentResult charge(BigDecimal amt) {
      // Stripe REST API call implementation
   }
}
```

QA VIEW & BENEFIT

Decouples clients from specific vendor implementations. Enables easy mock testing without real bank servers.

ABSTRACT CLASS

Template Pattern

Provides common state (connection pools, timeouts) and enforces a template workflow mixing concrete and abstract methods.

BaseSettlementEngine.java

```java
public abstract class BaseSettlementEngine {
   protected int timeoutMs = 5000; // State

   public void execute() { // Template method
      connect();
      processSettlement(); // Abstract step
      disconnect();
   }

   protected abstract void processSettlement();
}
```

REUSE & WORKFLOW ENFORCEMENT

Shares infrastructure state across subclasses while guaranteeing mandatory lifecycle execution order.

PILLAR 04

Polymorphism: Compile vs. Runtime

Dynamic Dispatch

SIMPLE DEFINITION

Polymorphism means "one name, many forms". It allows the same method name to work in different ways—either by giving it different inputs (Compile-time) or having subclasses provide their own custom implementation (Runtime).

STATIC (METHOD OVERLOADING)

Compile-Time Binding

Same method name, differing argument signatures within the same class. Compiler decides which method to call.

TransactionService.java

```java
// Overloaded variants
public class TransactionService {
   public void pay(BigDecimal amt) { ... }
   public void pay(BigDecimal amt, Currency fx) { ... }
   public void pay(BigDecimal amt, String memo) { ... }
}
```

RESOLUTION

Resolved at compilation via parameter types.

DYNAMIC (METHOD OVERRIDING)

Runtime Dynamic Dispatch

Subclass overrides parent method with identical signature. JVM invokes the specific override based on the concrete heap object.

PaymentGateway.java

```java
// Dynamic dispatch at runtime
PaymentGateway gateway = getGateway();

// Invokes PayPalGateway or StripeGateway at runtime
gateway.process(invoice);
```

QA VIEW & BENEFIT

Eliminates brittle switch statements across business flows.

Modern Java Features

Records: Immutable Data Carriers

Java 16+ Standard

SIMPLE DEFINITION

A Record is a quick, one-line Java class created solely to hold read-only data. Java automatically writes getters, constructor, and comparison methods behind the scenes, eliminating boilerplate code.

REAL-WORLD ANALOGY

Notarized Bill of Sale

A sealed receipt or vehicle transfer title. Once stamped, its fields (VIN, Sale Price, Date) cannot be rewritten with a pen. If changes occur, a new receipt must be produced.

IMMUTABILITY GUARANTEE

No Setters: Values are strictly read-only. Thread-safe by default.

BOILERPLATE REDUCTION

Replaces 60+ lines of verbose DTO boilerplate.

CODE IMPLEMENTATION

Concise & Immutable DTO

TransferRequest.java

```java
// One line replaces getters, equals, hashCode, toString
public record TransferRequest(
  String transactionId, BigDecimal amount,
  String senderIban, String receiverIban
) {
  // Compact validation constructor
  public TransferRequest {
    Objects.requireNonNull(transactionId);
    if (amount.compareTo(BigDecimal.ZERO) ⩽ 0) {
      throw new IllegalArgumentException("Invalid transfer amount");
    }
  }
}
```

QA VIEW & BENEFIT

Guarantees API test payloads cannot be mutated mid-flight.

Core Java Contract

equals() & hashCode() Contract

Critical Invariant

SIMPLE RULE

The Rule: If two objects are equal based on equals(), their hashCode() MUST return the exact same number. If you break this rule, HashMaps and HashSets will lose your data.

REAL-WORLD ANALOGY

Library Bookshelves

hashCode() gives the aisle number (Bucket). **equals()** verifies the exact title and ISBN on that specific shelf. If someone gives you the wrong aisle number, you will never find the book!

FINTECH BUG WARNING

FinTech Bug: If hashCode() is broken, duplicate payment idempotency checks fail, resulting in **double-charging customers**.

SYMMETRY MANDATE

Always override both equals() and hashCode() symmetrically.

CODE IMPLEMENTATION

IdempotencyKey Contract

IdempotencyKey.java

```java
public class IdempotencyKey {
  private final String key;

  @Override
  public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof IdempotencyKey that)) return false;
    return Objects.equals(this.key, that.key);
  }

  @Override
  public int hashCode() {
    return Objects.hash(this.key); // Same fields as equals
  }
}
```

QA VIEW & BENEFIT

QA View: Test that re-submitting an identical key returns HTTP 409 Conflict.

HANDS-ON LAB

Homework: Payment Gateway Interface

Practical Coding Exercise

ASSIGNMENT MISSION

Build a production-style, decoupled payment subsystem by implementing a shared Java Interface, two distinct payment vendor classes, an immutable Record payload, and a unit test validating dynamic dispatch.

SPECIFICATION REQUIREMENTS

4 Implementation Tasks

Task 1: Create a Java Record `PaymentReceipt(String id, BigDecimal amount, boolean success)`.

Task 2: Define interface `PaymentGateway` with method: `PaymentReceipt process(BigDecimal amount)`.

Task 3: Implement `StripeGateway` (2.9% fee deduction) and `PayPalGateway` (3.4% fee deduction).

Task 4: Write a test method that passes a list of `PaymentGateway` instances and processes $100.00 polymorphically.

STARTER TEMPLATE

Interface & Record Definition

PaymentGateway.java

```java
// Immutable receipt payload record
public record PaymentReceipt(
  String txId, BigDecimal netAmount, boolean success
) {}

// Shared payment interface contract
public interface PaymentGateway {
  PaymentReceipt process(BigDecimal amount);
}

// TODO: Implement StripeGateway and PayPalGateway!
```

BONUS CHALLENGE

Add an idempotency HashSet to verify deduplication.

DELIVERABLE

Submit your code and passing JUnit tests.



You Are Ready to Build Defensive, High-Reliability Software

Your background in testing gives you the ultimate developer advantage: you already know where code breaks. Use OOP principles not as academic theory, but as your toolkit to build bug-resistant applications.

Runtime Mastery
JDK, JVM, & Bytecode Mechanics

Core Pillars
Encapsulation, Inheritance, Abstraction, Polymorphism

Modern Java
Records & Contract Invariants